COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION: CPE21S1**	
DATE PERFORMED:	01/10/26	**DATE SUBMITTED:**	01/10/2026
NAME: Duenas Ralph Florenz		**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ HOA 10.1

1. PROCEDURE OUTPUT

ILOA

main.cpp

```cpp
#include <iostream>
#include "graph.h"


// graph implementation
int main() {
    // graph edges array
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph digraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex, weight):" << std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(digraph.head[i], i);
    }

    return 0;
}
```

graph.h

```h
#ifndef GRAPH_H
#define GRAPH_H

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode **head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node array
        head = new adjNode*[N]();
        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* current = head[i];
            while (current != nullptr) {
                adjNode* next = current->next;
                delete current;
                current = next;
            }
        }
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << " (" << i << ", " << ptr->val << ", " << ptr->cost << " ) ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

#endif
```

```
Graph adjacency list
(start_vertex, end_vertex, weight):
 (0, 2, 4 )  (0, 1, 2 )
 (1, 4, 3 )
 (2, 3, 2 )
 (3, 1, 4 )
 (4, 3, 3 )
```

ILOB

main.cpp

```cpp
#include <iostream>
#include "DFS.h"

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

DFS.h

```cpp
#ifndef DFS_H
#define DFS_H

#include <iostream>
#include <map>
#include <set>
#include <stack>
#include <string>
#include <vector>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges) {
            os << "(" << e.dest << ": " << e.weight << "), ";
        }
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph {
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N) {}

    // Return number of vertices in the graph
    auto vertices() const {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V) {
            edge_list.push_back(e);
        } else {
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }
```

```cpp
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph can be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

```cpp

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty()) {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end()) {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)) {
                // If the vertex hasn't been visited, insert it into the stack.
                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
auto create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges) {
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
void test_DFS() {
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order) {
        std::cout << v << std::endl;
    }
}

#endif
```

```
1:      (2: 0), (5: 0),
2:      (1: 0), (5: 0), (4: 0),
3:      (4: 0), (7: 0),
4:      (2: 0), (3: 0), (5: 0), (6: 0), (8: 0),
5:      (1: 0), (2: 0), (4: 0), (8: 0),
6:      (4: 0), (7: 0), (8: 0),
7:      (3: 0), (6: 0),
8:      (4: 0), (5: 0), (6: 0),

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\Documents\Duenas CPE010\ILOB>
```

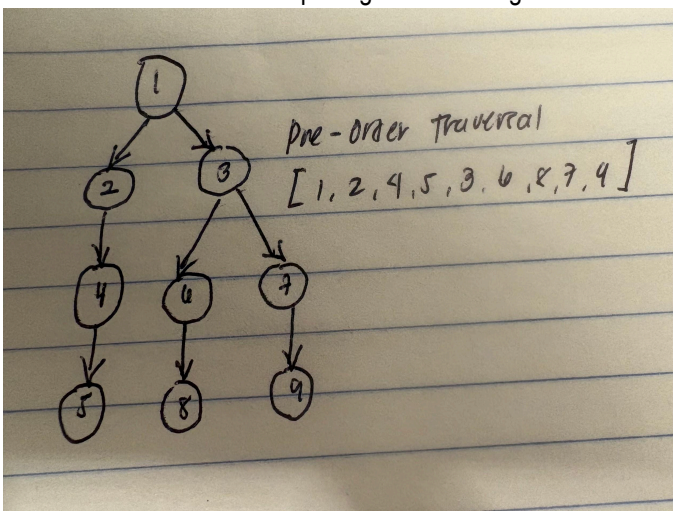
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- The Depth First Search (DFS) algorithm is the best choice to accomplish this task. It goes deep down one path first and backtracks to visit other paths.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- The tree traversal equivalent of Depth First Search is Pre-Order Traversal. This strategy visits the main node first before exploring its left and right branches.



In Pre-Order Traversal:

START

If current node is empty

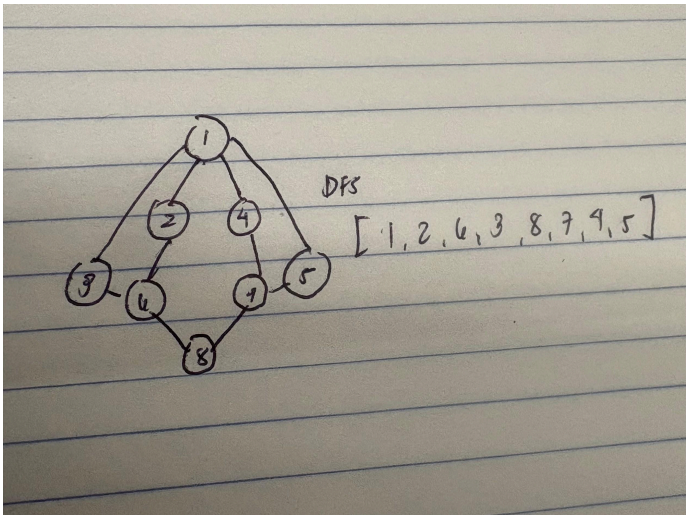
Stop

Show current node

PreOrder(left child)

PreOrder(right child)

END



In Depth-First Search (DFS):

Start

Create a stack

Put start node into stack

Mark start node as seen

While stack is not empty

Take top node from stack

Show current node

End

3 In the performed code, what data structure is used to implement the Breadth First Search?

- A Queue data structure is used to implement Breadth First Search. It helps process nodes step-by-step in a first-in, first-out order.

4 How many times can a node be visited in the BFS?

- A node can be visited only once during Breadth First Search. This ensures that nodes are not checked repeatedly and avoids getting stuck in loops.

Summary of lessons learned/ Analysis of the procedure/ Analysis of the supplementary activity

- This HOA showed me how a tree and a graph are different, where a tree stores items in a straight line and a graph connects items in many directions. In ILO A, I learned how to make a graph using a list of neighbors or a list of lines, as well as how to add new lines and count them all up. In ILO B, I learned the two ways to move through a graph: Depth-First Search (DFS) and Breadth-First Search (BFS). DFS works much like Pre-Order searching (root -> left -> right), but since a graph has no main top node, you just begin at any point you pick. Finally, BFS checks each point layer by layer to explore the full graph.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.